

Session : Principale
Régime : RM
Enseignant : Sofiane HACHICHA
Classe(s) : CPI2
Barème : 7 + 13

Date : 25/05/2024
Matière : Programmation Java 2
Durée : 90 minutes
Documents : Non autorisés
Nombre des pages : 7

Examen

Exercice 1

Vous êtes chargé de créer un système de gestion de bibliothèque. Voici les étapes à suivre :

1. Créez une classe **Livre** possédant les attributs privés suivants : titre (String), auteur (String), prix (double), anneePublication (int). Implémentez les méthodes nécessaires pour la sérialisation et la désérialisation des objets Livre en utilisant l'interface Serializable. Notez que les prix des livres ne seront pas inclus dans la sérialisation/désérialisation.

```
import java.io.Serializable;
```

```
public class Livre implements Serializable {
    private String titre;
    private String auteur;
    private transient double prix;
    private int anneePublication;
    public Livre(String titre, String auteur, double prix, int anneePublication) {
        this.titre = titre;
        this.auteur = auteur;
        this.prix = prix;
        this.anneePublication = anneePublication;
    }
    public String getTitre() {
        return titre;
    }

    public String getAuteur() {
        return auteur;
    }

    public double getPrix() {
        return prix;
    }

    public int getAnneePublication() {
        return anneePublication;
    }

    public String toString() {
        return "Livre [titre=" + titre + ", auteur=" + auteur + ", prix=" +
        prix + " ,anneePublication=" + anneePublication + "]";
    }
}
```

2. Créez une classe **Bibliothèque** contenant une liste de livres comme unique attribut. Utilisez une `ArrayList` initialement vide pour stocker les livres. Implémentez les méthodes suivantes :

- `ajouterLivre(Livre livre)` : Ajoute un livre à la liste de livres.
- `supprimerLivre(Livre livre)` : Supprime un livre de la liste de livres.
- `rechercherParTitre(String titre)` : Recherche un livre par titre et retourne une liste de livres ayant le même titre.
- `afficherLivres()` : Affiche tous les livres de la bibliothèque.

Ajoutez également deux méthodes à la classe `Bibliothèque` pour permettre la sauvegarde et le chargement de la bibliothèque, réalisant ainsi la sérialisation et la désérialisation des objets `Livre` :

- `sauvegarder(Path fichier)`
- `charger(Path fichier)`

```
import java.io.*;
import java.nio.file.*;
import java.util.*;

public class Bibliotheque implements Serializable {
    private List<Livre> livres = new ArrayList<>();

    public void ajouterLivre(Livre livre) {
        livres.add(livre);
    }

    public void supprimerLivre(Livre livre) {
        livres.remove(livre);
    }

    public List<Livre> rechercherParTitre(String titre) {
        List<Livre> resultat = new ArrayList<>();
        for (Livre livre : livres) {
            if (livre.getTitre().equals(titre)) {
                resultat.add(livre);
            }
        }
        return resultat;
    }

    public void afficherLivres() {
        for (Livre livre : livres) {
            System.out.println(livre);
        }
    }

    public void sauvegarder(Path fichier) {
        // try (ObjectOutputStream oos = new
        //ObjectOutputStream(Files.newOutputStream(fichier))) {

        try (ObjectOutputStream oos = new ObjectOutputStream(new
        FileOutputStream(fichier.toFile()))) {
            oos.writeObject(livres);
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

```

    public void charger(Path fichier) {
//try (ObjectInputStream ois = new
//ObjectInputStream(Files.newInputStream(fichier))) {

        try (ObjectInputStream ois = new ObjectInputStream(new
FileInputStream(fichier.toFile())) {
            livres = (List<Livre>) ois.readObject();
        } catch (IOException | ClassNotFoundException e) {
            e.printStackTrace();
        }
    }
}

```

3. Créez une classe principale **GestionBibliotheque** pour tester la bibliothèque. Ajoutez deux livres à la bibliothèque, sauvegardez puis chargez la bibliothèque à partir du fichier 'biblio.bin', situé dans le répertoire 'C:\Rep1' déjà existant. Assurez-vous de créer le fichier 'biblio.bin' s'il n'existe pas, puis affichez les livres de cette bibliothèque.
- ```

import java.io.IOException;
import java.nio.file.*;

```

```

public class GestionBibliotheque {
 public static void main(String[] args) throws IOException {
 Bibliotheque bibliotheque = new Bibliotheque();

 Livre livre1 = new Livre("Titre 5", "Auteur 1", 19.99, 2000);
 Livre livre2 = new Livre("Titre 8", "Auteur 2", 24.50, 2007);

 bibliotheque.ajouterLivre(livre1);
 bibliotheque.ajouterLivre(livre2);

 Path fichier = Paths.get("C:", "Rep1", "biblio.bin");
 if(! fichier.toFile().exists())
 fichier.toFile().createNewFile();

 bibliotheque.sauvegarder(fichier);

 bibliotheque.charger(fichier);
 bibliotheque.afficherLivres();
 }
}

```

## Exercice 2

Vous êtes chargé de développer une application de gestion des employés pour une entreprise. L'application doit permettre d'ajouter, de modifier, de supprimer et de visualiser les employés de l'entreprise. Pour ce faire, vous allez créer une interface graphique utilisant Swing pour interagir avec les employés et une base de données MySQL pour stocker les informations. Voici les étapes à suivre :

- Mettre en place une architecture MVC pour l'application.
- Créer une interface graphique en utilisant Swing pour les opérations CRUD (Create, Read, Update, Delete) sur les employés.
- Utiliser JDBC pour la connexion à la base de données MySQL et effectuer les opérations sur la table des employés.

**Aperçu des différentes classes :**

- a. La classe **Employee**, en tant qu'entité, modélise un employé au sein de l'entreprise, défini par un identifiant unique, un nom et un salaire.
- b. La classe **EmployeeModelDAO**, qui représente une classe combinée modèle et DAO, doit contenir les méthodes suivantes :
  - **establishConnection()** : Établit une connexion à la base de données au moment de l'exécution.
  - **createEmployeesTable()** : Crée la table employees si elle n'existe pas au moment de l'exécution.
  - **addEmployee(Employee emp)** : Ajoute un nouvel employé à la table employees.
  - **updateEmployee(Employee emp)** : Met à jour les informations d'un employé existant dans la table employees.
  - **deleteEmployee(int id)** : Supprime un employé, en fonction de son identifiant, de la table employees.
  - **getAllEmployees()** : Renvoie la liste de tous les employés de la table employees.
  - **idExists(int id)** : Vérifie si un identifiant existe déjà dans la table employees.
- c. La classe **EmployeeView**, qui représente la vue utilisant Swing, doit contenir les éléments suivants :
  - Des champs de texte (JTextField) pour saisir l'identifiant, le nom et le salaire d'un employé.
  - Des boutons (JButton) pour ajouter, modifier et supprimer les employés.
  - Une zone de texte (JTextArea), enveloppée dans un JScrollPane, pour afficher la liste des employés.
- d. La classe **EmployeeController**, qui agit comme médiateur entre la vue et la classe EmployeeModelDAO, doit contenir les méthodes suivantes :
  - **addEmployee(Employee emp)** : Appelé lorsque l'utilisateur souhaite ajouter un employé.
  - **updateEmployee(Employee emp)** : Appelé lorsque l'utilisateur souhaite modifier un employé.
  - **deleteEmployee(int id)** : Appelé lorsque l'utilisateur souhaite supprimer un employé.
  - **fetchEmployees()** : Appelé pour récupérer la liste des employés et les afficher dans la vue.
  - **idExists(int id)** : Appelé lorsque l'utilisateur souhaite vérifier si un identifiant existe déjà.
  - **updateView()** : Appelé pour rafraîchir ou mettre à jour la vue après des modifications suite à l'ajout, la mise à jour ou la suppression d'un employé.

**Scénarios d'utilisation :**

- **Ajouter un employé** : Lorsque l'utilisateur saisit les informations d'un employé dans les champs de texte et clique sur le bouton "Ajouter" :
  - Si l'identifiant n'existe pas, l'employé sera ajouté à la table employees, affiché dans la zone de texte, et une boîte de dialogue informative affichera le message : « Employé ajouté avec succès ».
  - Si l'identifiant saisi existe déjà, une boîte de dialogue d'erreur affichera le message : « ID existant, saisir un autre ID ».

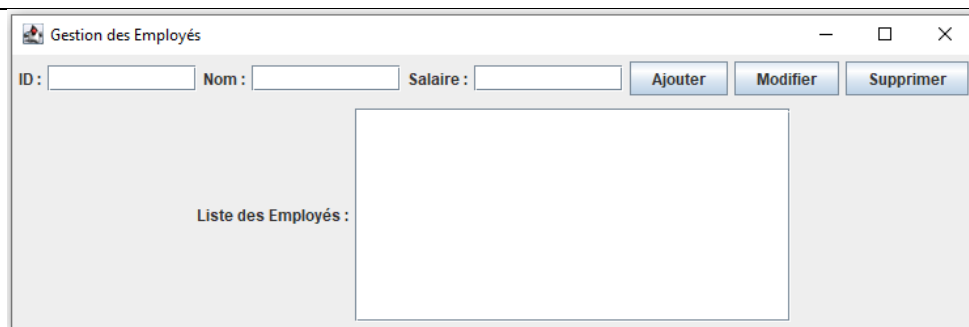
- **Modifier un employé** : Lorsque l'utilisateur clique sur le bouton "Modifier", une boîte de dialogue de saisie s'ouvre et affiche le message : « Saisir l'ID de l'employé à modifier » :
  - Si l'identifiant saisi existe, deux autres boîtes de dialogue permettent de saisir respectivement les nouvelles informations (nom et salaire) de l'employé ayant cet identifiant. Les informations de cet employé seront mises à jour dans la table employees, affichées dans la zone de texte, et une boîte de dialogue informative affichera le message : « Employé modifié avec succès ».
  - Si l'identifiant n'existe pas, une boîte de dialogue d'erreur affichera le message : « ID inexistant, saisir un autre ID ».
- **Supprimer un employé** : Lorsque l'utilisateur clique sur le bouton "Supprimer", une boîte de dialogue de saisie s'ouvre et affiche le message : « Saisir l'ID de l'employé à supprimer ».
  - Si l'identifiant saisi existe, l'employé sera supprimé de la table employees, retiré de la zone de texte, et une boîte de dialogue informative affichera le message : « Employé supprimé avec succès ».
  - Si l'identifiant n'existe pas, une boîte de dialogue d'erreur affichera le message : « ID inexistant, saisir un autre ID ».

**Au démarrage de l'application** : Tous les employés déjà ajoutés sont récupérés depuis la table employees et affichés dans la zone de texte.

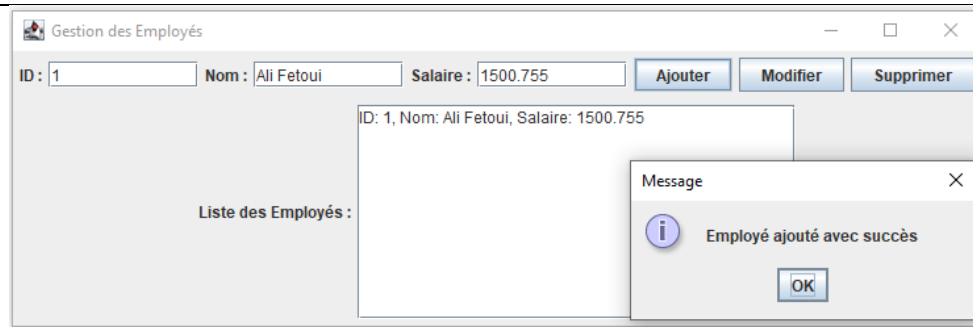
#### Recommandations :

- Utiliser les gestionnaires d'agencement appropriés pour organiser l'interface graphique. La méthode `setBounds()` ne doit pas être utilisée.
- Utiliser trois `.JTextField` vides de 10 colonnes chacun et un `JTextArea` vide de 10 lignes et 30 colonnes.
- Utiliser La commande SQL **"CREATE TABLE IF NOT EXISTS employees"** pour créer la table employees.
- La table employees doit comprendre les colonnes suivantes : **"id"** (entier, clé primaire), **"nom"** (chaîne de caractères) et **"salaire"** (décimal, précision de 7, échelle de 3).

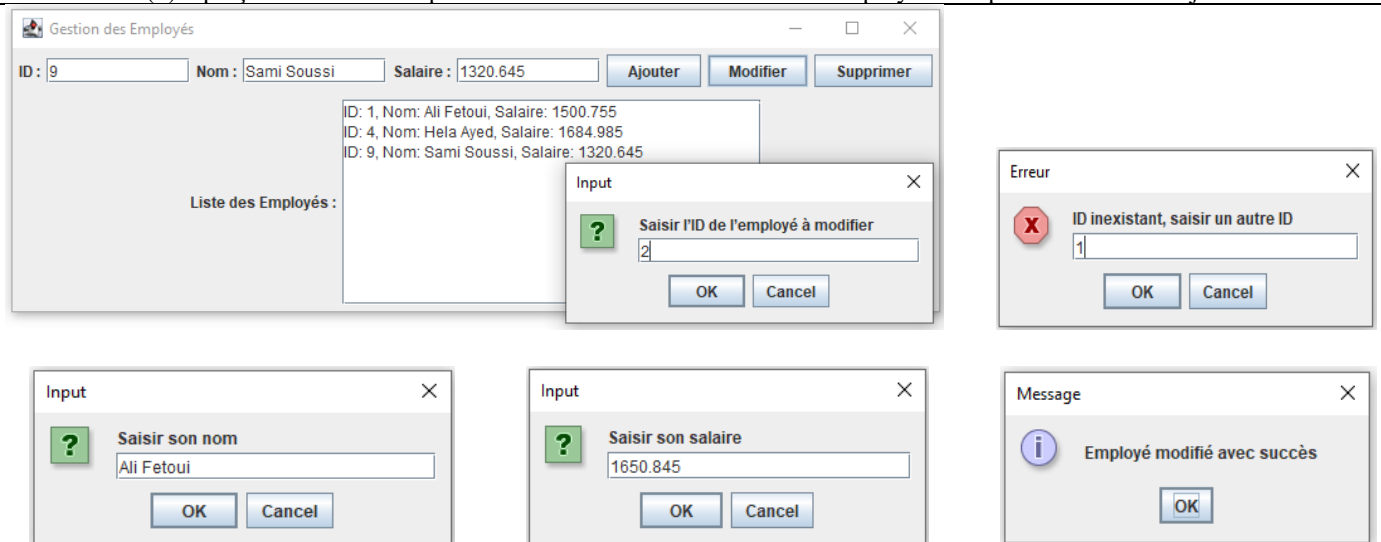
Le tableau suivant récapitule divers scénarios d'exécution de l'application.



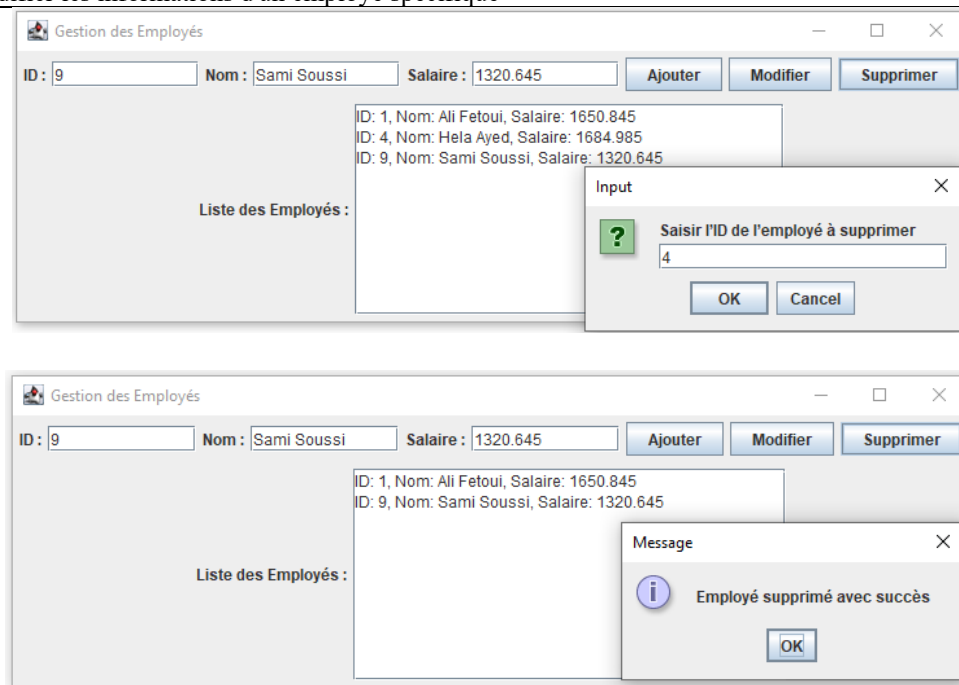
(a) Aperçu de l'interface graphique de l'application lors de son premier démarrage



(b) Aperçu de l'interface après avoir saisi les informations d'un employé et cliqué sur le bouton Ajouter



(c) Aperçu de l'interface après avoir cliqué sur le bouton Modifier, saisi un ID inexistant, puis saisi un ID existant suivi du nom et du salaire pour modifier les informations d'un employé spécifique



(d) Aperçu de l'interface après avoir cliqué sur le bouton Supprimer, saisi un ID existant pour supprimer un employé spécifique

Tableau – Scénarios d'exécution de l'application

Afin de vous guider dans le développement, les structures des différentes classes sont fournies ci-dessous. Il vous suffit de compléter les classes **EmployeeModelDAO**, **EmployeeController** et **EmployeeView**.

```
package model;
public class Employee {
 private int id;
 private String name;
 private double salary;

 public Employee(int id, String name, double salary) {
 this.id = id;
 this.name = name;
 this.salary = salary;
 }

 public int getId() {
 return id;
 }

 public void setId(int id) {
 this.id=id;
 }

 public String getName() {
 return name;
 }

 public void setName(String name) {
 this.name = name;
 }

 public double getSalary() {
 return salary;
 }

 public void setSalary(double salary) {
 this.salary = salary;
 }

 public String toString() {
 return "ID: " + id + ", Nom: " + name + ", Salaire: " + salary;
 }
}
```

```
package model;
import java.sql.*;
import java.util.*;
import java.math.BigDecimal;

public class EmployeeModelDAO {
 private List<Employee> employees;
 private Connection connection;
 private static final String DB_URL = "jdbc:mysql://localhost:3306/entreprise_db";
 private static final String DB_USERNAME = "cpi2";
 private static final String DB_PASSWORD = "isimg";
 public EmployeeModelDAO() {
 establishConnection();
 createEmployeesTable();
 }
}
```

```
private void establishConnection() {
 // A compléter
try {
 connection = DriverManager.getConnection(DB_URL, DB_USERNAME, DB_PASSWORD);
 } catch (SQLException e) {
 System.err.println("Erreur lors de la connexion à la base de données ");
 }
}

private void createEmployeesTable() {
 // A compléter
try {
 String sql = "CREATE TABLE IF NOT EXISTS employees (" +
 "id INT PRIMARY KEY," +
 "nom VARCHAR(255)," +
 "salaire DECIMAL(7,3))";
 connection.createStatement().executeUpdate(sql);
 } catch (SQLException e) {
 System.err.println("Erreur lors de la création de la table 'employees' : ");
 }
}

public void addEmployee(Employee emp) throws SQLException {
 // A compléter
PreparedStatement statement = connection.prepareStatement(
 "INSERT INTO employees (id, nom, salaire) VALUES (?, ?, ?)");
 statement.setInt(1, emp.getId());
 statement.setString(2, emp.getName());
 statement.setBigDecimal(3, new BigDecimal(emp.getSalary()));
 statement.executeUpdate();
}

public void updateEmployee(Employee emp) throws SQLException {
 // A compléter
 String updateSQL = "UPDATE employees SET nom = ?, salaire = ? WHERE id = ?";
 try (PreparedStatement pstmt = connection.prepareStatement(updateSQL)) {
 pstmt.setString(1, emp.getName());
 pstmt.setBigDecimal(2, new BigDecimal(emp.getSalary()));
 pstmt.setInt(3, emp.getId());
 pstmt.executeUpdate();
 }
}

public void deleteEmployee(int id) throws SQLException {
 // A compléter
 String deleteSQL = "DELETE FROM employees WHERE id = ?";
 try (PreparedStatement pstmt = connection.prepareStatement(deleteSQL)) {
 pstmt.setInt(1, id);
 pstmt.executeUpdate();
 }
}

public List<Employee> getAllEmployees() throws SQLException {
 // A compléter
 employees = new ArrayList<>();
 try (Statement stmt = connection.createStatement()) {
 ResultSet rs = stmt.executeQuery("SELECT * FROM employees");
 while (rs.next()) {
```



```
employees.add(new Employee(rs.getInt("id"), rs.getString("nom"),
rs.getBigDecimal("salaire").doubleValue()));
 }
}

return employees;
}

public boolean idExists(int id) throws SQLException {
 // A compléter
try (PreparedStatement pstmt = connection.prepareStatement("SELECT COUNT(*) FROM
employees WHERE id = ?")) {
 pstmt.setInt(1, id);
 ResultSet rs = pstmt.executeQuery();
 if (rs.next()) {
 return rs.getInt(1) > 0;
 }
}
return false;
}
}

package controller;
import java.sql.*;
import model.*;
import view.*;

public class EmployeeController {
 private EmployeeView view;
 private EmployeeModelDAO model;

 public EmployeeController(EmployeeView view, EmployeeModelDAO model) {
 this.view = view;
 this.model = model;
 }

 public void addEmployee(Employee emp) throws SQLException {
 // A compléter
 model.addEmployee(emp);
 updateView();
 }

 public void updateEmployee(Employee emp) throws SQLException {
 // A compléter
 model.updateEmployee(emp);
 updateView();
 }

 public void deleteEmployee(int id) throws SQLException {
 // A compléter
 model.deleteEmployee(id);
 updateView() ;
 }

 public void fetchEmployees() throws SQLException {
 // A compléter
 StringBuilder sb = new StringBuilder();
 for (Employee employee : model.getAllEmployees()) {
 sb.append(employee).append("\n");
 }
 }
}
```

```
 view.displayEmployees(sb.toString());
 }

 public boolean idExists(int id) throws SQLException {
 // A compléter
 return model.idExists(id);
 }

 private void updateView() throws SQLException {
 fetchEmployees();
 }
}

package view;
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
import java.sql.*;
import controller.*;
import model.*;

public class EmployeeView implements ActionListener {
 private JFrame f;
 private JTextField idField, nameField, salaryField;
 private JButton addButton, updateButton, deleteButton;
 private JTextArea displayArea;
 private EmployeeController controller;
 private Employee employee;

 public EmployeeView() {
 // A compléter
 f = new JFrame("Gestion des Employés");
 f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

 idField = new JTextField(10);
 nameField = new JTextField(10);
 salaryField = new JTextField(10);

 addButton = new JButton("Ajouter");
 updateButton = new JButton("Modifier");
 deleteButton = new JButton("Supprimer");
 displayArea = new JTextArea(10, 30);
 displayArea.setEditable(false);

 addButton.addActionListener(this);
 updateButton.addActionListener(this);
 deleteButton.addActionListener(this);

 JPanel inputPanel = new JPanel();
 inputPanel.add(new JLabel("ID :"));
 inputPanel.add(idField);
 inputPanel.add(new JLabel("Nom :"));
 inputPanel.add(nameField);
 inputPanel.add(new JLabel("Salaire :"));
 inputPanel.add(salaryField);
 inputPanel.add(addButton);
 inputPanel.add(updateButton);
 inputPanel.add(deleteButton);

 JPanel displayPanel = new JPanel();
```

```
displayPanel.add(new JLabel("Liste des Employés :"));
displayPanel.add(new JScrollPane(displayArea));

f.getContentPane().add(inputPanel, BorderLayout.PAGE_START);
f.getContentPane().add(displayPanel, BorderLayout.CENTER);

f.pack();
f.setLocationRelativeTo(null);
f.setVisible(true);
}

public void actionPerformed(ActionEvent e) {
 // A compléter
 if (e.getSource() == addButton) {
 try {
 if (controller.idExists(Integer.parseInt(idField.getText()))) {
 JOptionPane.showMessageDialog(null, "ID existant, saisir un autre ID", "Erreur",
 JOptionPane.ERROR_MESSAGE);
 return;
 }
 employee = new Employee(Integer.parseInt(idField.getText()), nameField.getText(),
 Double.parseDouble(salaryField.getText()));
 controller.addEmployee(employee);

 } catch (SQLException e1) {
 e1.printStackTrace();
 }
 JOptionPane.showMessageDialog(null, "Employé ajouté avec succès");
 } else if (e.getSource() == updateButton) {

 int n = Integer.parseInt(JOptionPane.showInputDialog(null, "Saisir l'ID de
 l'employé à modifier"));
 try {
 while (!controller.idExists(n)) {
 n = Integer.parseInt(JOptionPane.showInputDialog(null, "ID inexistant, saisir un
 autre ID", "Erreur", JOptionPane.ERROR_MESSAGE));
 }
 String nom = JOptionPane.showInputDialog(null, "Saisir son nom");
 Double sal = Double.parseDouble(JOptionPane.showInputDialog(null,
 "Saisir son salaire"));

 employee = new Employee(n, nom, sal);
 controller.updateEmployee(employee);
 } catch (SQLException e1) {
 e1.printStackTrace();
 }

 JOptionPane.showMessageDialog(null, "Employé modifié avec succès");
 } else if (e.getSource() == deleteButton) {
 int n = Integer.parseInt(JOptionPane.showInputDialog(null,
 "Saisir l'ID de l'employé à supprimer"));
 try {
 while (!controller.idExists(n)) {
 n = Integer.parseInt(JOptionPane.showInputDialog(null, "ID
 inexistant, saisir un autre ID", "Erreur", JOptionPane.ERROR_MESSAGE));
 }
 controller.deleteEmployee(n);
 } catch (SQLException e1) {
 e1.printStackTrace();
 }
 JOptionPane.showMessageDialog(null, "Employé supprimé avec succès");
 }
}
```

```
 }
}

public void setController(EmployeeController controller) {
 this.controller = controller;
}

public void displayEmployees(String employeesText) {
 displayArea.setText(employeesText);
}

public static void main(String[] args) {

 EmployeeModelDAO model = new EmployeeModelDAO();
 SwingUtilities.invokeLater(() -> {
 try {
 EmployeeView view = new EmployeeView();
 EmployeeController controller = new EmployeeController(view, model);
 view.setController(controller);
 controller.fetchEmployees();
 } catch (SQLException e) {
 e.printStackTrace();
 }
 });
}
```

**BON TRAVAIL**